Sarah Schultz sschul15 slegitim
Sybille Legitime s0s sardonyx

Project – I

Introduction

Boolean Satisfiability (SAT) was the first problem classified as NP complete. Given a Boolean formula, SAT determines if there is a way to assign binary variables such that the formula evaluates to True. Input typically takes the form of an equation in Conjunctive Normal Form (CNF), which conjoins clauses together. Clauses are disjoined by literals. Therefore, for a formula to be "satisfiable" (SAT), all clauses must be true. For a formula to be "unsatisfiable" (UNSAT), one clause must be false. SAT problems have many applications in the real world, especially in hardware and software verification, electronic design automation, logistics, and planning.

Implementation Strategy

In this project, we were tasked with implementing a SAT solver based on the DPLL algorithm that would test various problem instances in the form of CNF files and determine if the instance was SAT or UNSAT in five minutes. Before implementing any heuristics, we chose to create a working version of the DPLL algorithm, no matter how time-consuming or inefficient it was. Our chosen programming language was Python for its ease of use. Although Python is a lot slower as an interpreter than C or Rust as compiled languages, we reasoned that we could write in Python first, and if time allowed and we needed faster execution, translate to C. After implementing a baseline algorithm, we implemented a simple heuristic to choose the literal to branch on based on the number of occurrences of the literal in clauses that had not been satisfied yet. This represented our initial submission, which solved 9 out of 21 test instances.

Many SAT heuristics developed recently were promising, including Conflict Driven Clause Learning, watched literals, and VSIDS. However, all of these require significant programming effort and would be a different algorithm from the assigned DPLL. However, even within DPLL, there is a huge design space. We experimented with several different heuristics and different parameters within these heuristics, which will be discussed later in the report. For the final implementation, we chose one of the simpler heuristics, which was Dynamic Largest Combined Sum (DLCS). DLCS finds the literal with the most occurrences in unresolved clauses and chooses it to branch on. Our version also assigned a polarity based on if the literal was present more often as a positive or negative value. This version solved 13 of the 21 test instances.

After testing other heuristics alongside DLCS and comparing the results, we found that DLCS consistently showed the best results. However, many of our instances were timing out, especially those that were much larger. We decided to focus on speeding up the code. Because of time constraints, we opted to first explore a more optimal version of the Python code, rather than completely translate to C. After implementing some optimizations, such as using arrays rather than dictionaries, eliminating unnecessary uses of the copy() function, and only evaluating unsatisfied clauses, we were able to achieve much faster execution times, although there were still some instances that were unable to be solved.

Overall, we estimate that we spent between 30 and 40 hours on this project.

Other Techniques and Observations

In addition to implementing DLCS, we tried several other heuristics. One of these was randomized DLCS, which reduces greediness in the heuristic by choosing between the top x candidates randomly. This value "x" is chosen by the user, and with more time, we would have loved to explore what values give better results. In our version, we chose to use a value of 4, but in hindsight, we realize the value should most likely vary between instances. Some instances in the test set had only 5 variables, and randomly choosing between 4 of them would not have been very beneficial. However, some instances in the test set had thousands of variables; choosing between 4 of them was more meaningful.

We attempted Bohm's algorithm, which tries to satisfy the smallest number of clauses. Bohm's is costly, especially for extremely large test instances. When instances had thousands of variables, a single pass of Bohm's algorithm could take over a minute, which was not ideal. To reduce this, we decided to randomly sample the number of clauses to evaluate in the algorithm. This was helpful for instances with hundreds of thousands of clauses. We also decided to randomly sample from the variables within these clauses, so in instances with thousands of variables, we would only have to look at a fraction of them. The sample size for both the clause size and variable size was tweaked a little bit, but it is another area where finding the optimal sampling rate would have been beneficial.

Before implementing most of the heuristics we tried, we profiled the input CNF files to look at the number of variables, clauses, and how many clauses of different lengths existed in each file. We decided to implement a heuristic that focused on shorter clause lengths because several instances had an overwhelming amount of smaller clause lengths (2 or 3 literals). We assumed that Bohm's algorithm would be most beneficial for these cases. However, we found that Bohm's algorithm was most helpful for instances without variation in clause length (i.e., all clauses were the same size). These instances had struggled with DLCS. Additionally, several instances were solved by DLCS, not by Bohm's. Because of this, we figured there was some trade-off between the two heuristics, and we hoped to identify it and leverage it to solve as many instances as possible.

One possibility for combining the heuristics was to use a counter to count how many times we branched, and to use Bohm's algorithm every "x" branches. Every other time, use DLCS, since it is less expensive. Another possibility was to use Bohm's algorithm for the first "x" literals branched on, and for the rest of the algorithm, use DLCS. The motivation behind this was that a bad branching decision earlier on can more significantly impact the decision tree, so it would be better to use the more costly heuristic earlier in hopes that it would be better for the overall result. Again, with more time, it would have been great to be able to sweep the design parameters for these heuristics, but with the parameters we were able to test, we were not able to strike a balance between Bohm's and DLCS, and the results did not outperform regular DLCS.

Similar to Bohm's algorithm, we also attempted to use MOMS (Maximum Occurrences in clauses of Minimum Size) as our selection heuristic. MOMS was similar in intuition to Bohm's with a slightly different implementation. MOMS had promising results, reducing execution times for several instances by 4x, but in the end, it timed out more overall than the DLCS heuristic. With more time, we would have wanted to try alternating between MOMS and DLCS and explore their parameters more.